case-study-fullstack-sdet-candidate-brief

Case Study: Fullstack Engineer (SDET Depth)

For: shortlisted candidates. **Format:** take-home + live defense. **Duration:** 3 to 4 days.

Welcome, and thank you for getting this far. This document plus the platform repo you get access to are **everything** you need: there is no separate back-and-forth or live Q&A. Read this once fully before you start. If something is ambiguous, treat that as part of the exercise: make a reasonable assumption, write it down, and proceed.

The role, in short

You would own the **quality net** for a client-delivery squad: the system that makes a missing spec, a dropped requirement, or a silent regression hard to ship unnoticed. You are a fullstack engineer who also builds quality in, from the first sprint, not a tester bolted on at the end. You build your own features *and* you own the system that keeps the whole team's work reliable.

A large part of the job is hardening the **development workflow**, not just the code: making sure no change moves forward without its inputs, a spec or PRD, acceptance criteria or test scenarios, and a solution/design plan. When those inputs are missing, the work is not ready, and your system is what makes that visible and enforced. We cannot always make the people upstream write a clean spec, so the engineering gate is where a solid spec becomes a hard precondition.

This case study is a realistic slice of that job.

What we are really looking at

Be relaxed about one thing: **we are not grading how much test code you produce.** You have full AI access and we expect you to use it. Anyone can generate a thousand lines of passing tests in an afternoon, and that is not the skill we are hiring for.

We are looking at the **calls you make**:

- Can you look at a real codebase and tell us what is wrong, what is missing, and what is risky?
- Can you harden the *workflow* so work cannot move forward without its inputs (a spec, acceptance criteria, a design plan), not just catch bugs after they are built?
- Can you build a quality system that catches the *next* problem, not just patch this one?
- When something is about to ship, can you judge whether it is actually safe, and own that decision?

Strong judgment with modest code will beat impressive code with weak judgment. Work the way you would on a real team.

Setup: you work in your own repo (read this carefully)

You do not depend on us for anything except access to one source repo. The platform is two services, an **API** and a **web app**, that live together in a single repo as `api/` and `web/` folders. Here is the whole flow:

1. **We share the platform repo.** We give you read access to one GitHub repository:

> **Platform repo:** `https://github.com/rakamindev/ai-interview-platform`

1. It holds both services (`api/` and `web/`), each with a `README.md` explaining how to run it locally; you run them together.
2. **Make your own copy, de-linked from ours.** Do **not** fork it (a fork keeps a visible link back to the source). Instead, copy it into a fresh **public** repo on your own account so it is fully yours with clean history. From a terminal:

```
git clone https://github.com/rakamindev/ai-interview-platform platform
cd platform
rm -rf .git
git init && git add . && git commit -m "Initial import of the platform"
```

```
git branch -M main
git remote add origin [YOUR_REPO_URL]
git push -u origin main
```

1. Create `[YOUR_REPO_URL]` first as an empty **public** repo on your GitHub or GitLab, named per the naming note below. This gives you an independent repo with no link back to us, and nothing for either side to set up or manage.
2. **Work in your repo for the whole case study.** Commit as you go on `main` for the bulk of the work. The one exception is your **workflow gate**: it has to be *demonstrated*, not just described. Open at least two pull requests against your own repo so the gate runs on a real change:
   - one PR the gate **blocks** (for example, a change with no linked spec or acceptance criteria, or no test), and
   - one PR the gate **passes** (inputs linked, tests present).
3. Leave both PRs visible (open, closed, or merged) so we can see the gate firing red and green by itself. Everything else can stay as direct commits to `main`. Put every written document we ask for in an `/assessment` folder at the root; code changes go in `api/` or `web/`.
4. **At the deadline, submit your repo URL.** Submit the link to your public repo through the same platform where you received this brief. Because the repo is public, we can review it right away, no invites, no access requests, nothing to set up.

That is the entire mechanism. We share one source repo, you work in your own public copy, you submit the link through the platform at the end.

Naming note (please follow this)

Your repo is public, so:

- **Pick a unique repo name of your own** (for example, include your own handle, like `yourname-quality-net`), not a generic guessable one. This keeps your work from being trivially found and copied by other candidates.
- **Keep it neutral.** A name about quality engineering, not about us or any client, reads better as a portfolio piece anyway.

Our quality bar (read this, it is how we score)

We are explicit about the bar, because quality *is* the job. These principles define it.

1. Work needs inputs (the Definition of Ready). A change is not ready to build, and a feature is not “done,” without its inputs present: a spec or PRD, acceptance criteria or test scenarios, and a short solution/design plan. This is our real-world core problem, work that starts from a ghost spec and rots from there. Your system should make missing inputs **block** the work, not pass silently. Since we cannot always force a clean spec upstream, the engineering gate is where a solid spec becomes a hard precondition.

2. Correctness is about data and outcome, not the screen. A feature that shows the right thing while storing or computing the wrong thing underneath is a serious defect, not a cosmetic one. Always check what is persisted and what is computed, not only what is rendered.

3. We use one severity language. Use it too, when you rank risks (task 1) and gate releases (task 3). Apply top to bottom, stop at the first match:

Severity	Meaning
P0 Blocker	The objective cannot be achieved at all, no workaround. The main function is broken.

P1 Major	It looks like it works, but the data or logic is wrong underneath; or the objective is reachable only with a manual workaround. **Any data-integrity issue is at least P1.**
P2 Minor	It works and the data is correct, but there is a limited, non-blocking issue.
P3 Cosmetic	Purely visual or copy. No function or data impact.

4. Honesty beats green. A known risk you disclose, with a mitigation and a named owner, is professional. A risk you hide behind a passing build is the failure mode we most want to screen out. We will trust a well-argued "blocked" more than a suspicious "all green."

5. Green is earned, not forced. When a check goes red, you make it green by **fixing the defect**, never by deleting or weakening the check. Turning red to green by gutting the test ends the exercise. We read your commit history.

And throughout: we value a **small, sharp, reproducible** system over broad shallow coverage. A few checks that each catch a real class of failure beat a thousand trivial tests.

The bigger picture: where your gate sits (six stages, four gates)

So you understand *why* the inputs matter, here is the delivery model your role lives inside. Every engagement runs six stages, and four gates control what is allowed to move between them. Each gate is co-signed by two roles, so no one passes their own work downstream unchecked.

The six stages (S0 to S5):

Stage	What happens	Key artifacts produced
S0 Qualify & shape	Is this worth doing, can we deliver it?	opportunity brief
S1 Frame & scope	Turn the mandate into the *real* problem; cut scope into modules	problem statement, scope & module map, out-of-scope list
S2 Specify & prototype	Turn scope into buildable artifacts	PRD, business rules, **acceptance criteria**, clickable prototype, design, a backlog with a Definition of Done per task
S3 Build & validate	Build with quality from sprint one	working modules, passing checks, traceability matrix
S4 Ship & adopt	UAT against the real problem, go-live, training	UAT sign-off, adoption

S5 Prove & expand	Outcome review, lessons, next phase	outcome evidence

The four gates (G1 to G4):

Gate	After	The question	What it needs
G1 Scope lock	S1	Are we solving the right problem, inside a defensible boundary?	confirmed problem statement, scope, explicit out-of-scope
G2 Buildable	S2	Can the client click it, and can engineering build it?	approved prototype, Given/When/Then **acceptance criteria**, a Definition of Done on every task
G3 Release-ready	S3	Does it work, and does it still trace to the need?	all DoDs met, **regression green**, **traceability complete both ways**
G4 Outcome	S4	Did it solve the problem, and will they use it?	UAT validates the outcome, not just that features function

Where you fit. You own the engineering quality gate. The inputs the bar above talks about (a spec/PRD, acceptance criteria or test scenarios, a solution/design plan) are the outputs of **S1 to S2** and the entry condition for **G2**. Your net is what makes **G2** ("ready to build") and **G3** ("ready to release") real, automatic, and impossible to skip. In the real world we cannot always force S1 to S2 to be done well upstream, so **your gate is where we require that they were**, no inputs, no build; no regression-green and traceability, no release. That enforcement is the workflow hardening this case study is really about.

What to do

You have 3 to 5 days to work on this, from when you get access until your deadline (the exact dates are shown to you on the platform). Manage your own time across the work; we are not looking for nights-and-weekends heroics, we are looking at how you prioritize when time is finite.

The work is an end-to-end loop, the loop this role lives in: **assess → build the gate → fix to green → release.** You find the problems, build the net that catches them (it goes red), fix them as the engineer who owns the code, watch the net go from red to green on real fixes, then cut a gated release. Plus a short live defense at the end. Every written deliverable is a markdown file in the `/assessment` folder of your repo; every code change is committed to the platform itself.

1. Assess the platform

Go through the platform, the API and the web together, and tell us its real state: what works, what does not, what is missing, and what would worry you if this went in front of a client tomorrow. Bugs that live in the seam between the two services (the web shows one thing, the API stored another) are exactly the kind worth finding.

Done when:

- Every risk has a severity (P0-P3, per the bar above), a one-line impact (who or what is hurt), and how you found it (repro steps or evidence).
- The list is ranked, not flat.
- You separate *missing or ambiguous spec* (we never defined it) from *built wrong* (we defined it and the build does not match).

- Missing **inputs** are first-class findings, not just code bugs: a feature with no PRD, no acceptance criteria, or no design plan is a risk in its own right, because nobody can say whether it is correct.
- You looked past the happy-path UI: data integrity and error/edge states are covered.

What great looks like:

- You name the systemic pattern behind several issues, not just the issues ("this class of bug recurs because…").
- You draw a clear ship / do-not-ship line and say what you would gate before this reaches a client.
- Someone who has never seen the app understands the real risk in five minutes.

Deliver: `/assessment/01-audit.md`

2. Build the net

Build the checks, gates, and automated coverage you would want a real team to rely on. This is two halves: the **workflow gate** that requires a change to carry its inputs before it proceeds, and the **test/CI net** that catches defects in the code. Document it so another engineer could run and extend it from your notes.

As on any real engagement, the repo carries project context beyond the code (notes, tickets, requests, release notes). Look around, not just at the source, and handle anything you find the way you would in production.

Done when:

- Your net enforces a **Definition of Ready / Done**: a change cannot proceed (and a feature is not "done") without its inputs linked, a spec or PRD, acceptance criteria or test scenarios, and a short solution/design plan. No inputs, no green. This is the part that hardens the *workflow*, not just the code.
- CI runs on every change and goes **red on the real issues you found in step 1** (this is what step 3 then fixes). It goes red by itself, no human needed.
- The gate blocks at least: a change with no test, a change with no linked spec/acceptance criteria, and a regression on a critical path.
- Coverage targets the risk-carrying paths, including data integrity, not a coverage percentage on trivial code.
- A short README lets another engineer run and extend the net.

What great looks like:

- The workflow gate is real and usable: a clear, enforced way (a PR template plus a CI check, a Definition-of-Ready checklist that actually blocks) that makes "no spec, no acceptance criteria" impossible to merge silently, without becoming bureaucratic.
- Small and sharp: a few checks, each catching a real class of failure, beats a thousand trivial tests.
- The net would have caught the issues you found in step 1 **before a human looked**.
- You can say what each check protects and what it deliberately does not cover.

Deliver: your code changes committed to the platform, plus a short write-up at `/assessment/02-quality-system.md` explaining what you built, how to run it, and what it protects.

3. Fix to green

Now switch hats to the engineer who owns the code. Fix the real defects your net is flagging, starting with the **P0 and P1** issues from your audit (you do not need to fix every P2/P3 in the time you have). Each fix should flip its check from **red to green by correcting the defect**, not by weakening the check.

This is the fullstack half of the role: you do not just catch problems, you ship the fix across the stack (data, API, UI) and let the net prove it.

Done when:

- The P0/P1 defects you identified are fixed with real code changes.
- The checks for those defects now pass, and your history shows the red-then-green transition (a failing run before the

fix, a passing run after).

- No check was made green by deleting or weakening it. (We read the diff.)

What great looks like:

- The fix addresses the root cause, not the symptom, and you say so in the commit or notes.
- You add or keep a regression check so the same defect cannot silently return.
- Anything you chose not to fix is a conscious, stated call, not an oversight.

Deliver: your fixes committed to the platform, and a short red-to-green note inside `/assessment/02-quality-system.md` (what was red, what you changed, what is green now).

4. Cut a release and gate it

Now play the release yourself. Treat **your improved version** as a release about to ship, and put a real release process around it:

- **Tag a release** (for example `v1.0.0`) and write a `RELEASE_NOTES.md` (or `CHANGELOG.md`) stating what this version claims to deliver.
- **Wire CI to gate the release.** On the tag (or a release event), your pipeline runs your net and surfaces a clear **release status**: green only if your quality gates pass, blocked otherwise. The status should be tied to the tag, so anyone can see whether that version is releasable.
- **Make the call.** Based on what your gate shows, decide whether this release is actually **ready to ship or should be blocked**, and write your decision: what your gate checked, what it found, and your recommendation. The hard part is judging your **own** work honestly: if any P0 or P1 is still open, the honest status is blocked, or shipped only with explicit, documented risk acceptance and a named owner.

This is the real job in miniature: not “do the tests pass,” but “is this version safe to release, and does the pipeline prove it.”

Done when:

- A tag and release notes exist; the notes state what the version claims to deliver.
- CI triggers on the tag and produces an unambiguous **releasable / blocked** status tied to that tag.
- If any P0/P1 remains, your decision says so and blocks (or ships only with explicit, documented risk acceptance).
- Your decision records: what the gate checked, what it found, your recommendation, and who owns the risk if it ships.

What great looks like:

- The status is legible to a non-engineer: “this version is releasable / blocked” without reading logs.
- You block on a real data or logic risk even when everything looks fine; or if you ship, you disclose the risk honestly with a mitigation. Hiding a known risk fails.
- The gate is reusable for the next release, not a throwaway script.

Deliver: `/assessment/03-release-decision.md`, plus the tag, the release notes, and the release-gating CI committed to your repo.

5. Defend it (live)

A 60 to 90 minute video session with the CTO (and Tech Lead if available), scheduled with you after you submit. We walk through your work together and ask you to talk us through your reasoning and your decisions. This is a conversation, not an interrogation. We want to understand how you think. Nothing to prepare or submit; just be ready to discuss what you did.

What to submit (checklist)

Everything lives in your own public repo. By the deadline, make sure your `main` branch has:

- [ ] `/assessment/01-audit.md` — your assessment and severity-ranked risk list (with each item’s final status: fixed or remaining)

- [ ] Your net (the quality system) committed to the platform
- [ ] **Two pull requests demonstrating your workflow gate**, one it blocks, one it passes, left visible
- [ ] Your fixes committed, with the **red-to-green transition visible in the history**
- [ ] `/assessment/02-quality-system.md` — what the net is, how to run it, what it protects, and the red-to-green story
- [ ] `/assessment/03-release-decision.md` — your gated release of the improved version, with the ship/block decision
- [ ] Anything else you built or noted along the way (optional, welcome)

Then submit your public repo URL through the platform to say you are done. Commit messages and small notes-to-self are welcome, we like seeing how you worked, not just the final state.

How we will evaluate

Each task carries its own **Done when** and **What great looks like**; that is the literal bar, and the quality-bar section above is the lens. Above those specifics, two things weighted equally:

1. **Can you build and ship?** The quality net, the checks, the release gate, *and* the fullstack fixes that take the net from red to green across the stack. Judged against "small, sharp, reproducible," not lines of test code, and green must be earned by fixing, never by gutting a check.
2. **Do you make sound calls?** Spotting what is missing or wrong, ranking it in our severity language, and deciding what ships, honestly, including about your own work, with reasons you can stand behind.

We need both. A candidate strong on one and weak on the other is not yet a fit for this specific seat, and that is fine, it is a specialized role. "Done" clears the bar; "great" is what we are hoping to see.

Ground rules and logistics

- **AI is encouraged.** Use it however you normally work. We will ask you what it got right and where you had to correct it, so use it the way a senior engineer does: as leverage you verify, not an oracle you trust blindly.
- **Time:** you have 3 to 4 days to work on this. We are not looking for nights-and-weekends heroics; we are looking for how you prioritize when time is finite.
- **No live Q&A, by design.** There is no back-and-forth channel during the case study. If something is ambiguous or missing, make a reasonable call, write the assumption into your `/assessment` notes, and keep moving. Working through ambiguity is part of the job, and we read your assumptions as signal.
- **Confidentiality:** your repo can be public (treat it as a portfolio piece), but do not name our company, the product, or any client anywhere in it (see the naming note above). That is the one hard rule.

A note before you start

The platform you will see is realistic, which means it is imperfect, the same way real client work is. That is intentional. We are not handing you a clean exercise with one right answer; we are handing you a real situation and asking how you would make it trustworthy. There is no single correct path. Show us your judgment.

Good luck. We are genuinely looking forward to seeing how you think.

What you have

So you know nothing is missing, here is everything involved. It is all in your hands the moment you start; the live defense afterward is scheduled for you through the platform.

Item	When	What it is
This brief	Now	Everything you need to do the work, self-contained

https://github.com/rakamindev/ai-interview-platform	At the start	Read access to the platform repo (`api/` + `web/`), with run instructions inside
Submit your repo URL	At your deadline	Through the same platform where you received this brief

There is no separate question channel. If this document does not answer something, treat it as part of the exercise: make a reasonable assumption, note it in your `/assessment` files, and proceed.